Taller 1: Programación Estructurada y Archivos en C

Introducción a la Programación Embebida

Docente: Carlos Andrés Erazo Garzón
carlos-erazo@javeriana.edu.co

Estudiante:

Juan Pablo Cañón Contreras

Departamento de Electrónica
Pontificia Universidad Javeriana

Semestre 2026-1

9 de marzo de 2026

Índice

1. Introducción **3**

2. Punto 1: Aproximación de π con la serie de Leibniz **3**
2.1. Descripción del problema . . . 3
2.2. Funciones implementadas . . . 3
2.3. Validación de datos . . . 3
2.4. Código fuente . . . 3
2.5. Casos de prueba . . . 5

3. Punto 2: Cifrado César **5**
3.1. Descripción del problema . . . 5
3.2. Funciones implementadas . . . 5
3.3. Validación de datos . . . 5
3.4. Código fuente . . . 6
3.5. Casos de prueba . . . 7

4. Punto 3: Filtro de promedio móvil (suavizado de señal) **7**
4.1. Descripción del problema . . . 7
4.2. Funciones implementadas . . . 8
4.3. Validación de datos . . . 8
4.4. Código fuente . . . 8
4.5. Casos de prueba . . . 9

5. Punto 4: Detección de picos en una matriz 2D **10**
5.1. Descripción del problema . . . 10
5.2. Funciones implementadas . . . 10
5.3. Validación de datos . . . 10
5.4. Código fuente . . . 10
5.5. Casos de prueba . . . 12

6. Punto 5: Lectura de archivo con estructuras **12**
6.1. Descripción del problema . . . 12
6.2. Funciones implementadas . . . 12
6.3. Estructura de datos . . . 12
6.4. Validación de datos . . . 13
6.5. Código fuente . . . 13
6.6. Casos de prueba . . . 14

7. Punto 6: Análisis de temperaturas **14**
7.1. Descripción del problema . . . 14
7.2. Funciones implementadas . . . 15
7.3. Validación de datos . . . 15
7.4. Código fuente . . . 15
7.5. Casos de prueba . . . 17

8. Punto 7: Exportar datos corregidos a archivo **17**
8.1. Descripción del problema 17
8.2. Funciones implementadas 17
8.3. Validación de datos 18
8.4. Código fuente 18
8.5. Casos de prueba 19

9. Conclusiones **20**

1. Introducción

El presente informe documenta el desarrollo del Taller 1 de la asignatura *Introducción a la Programación Embebida*. El objetivo principal es aplicar los conceptos de tipos de datos, cadenas de caracteres, arreglos (1D y 2D), estructuras (`struct`) y persistencia de datos (lectura y escritura de archivos con `sscanf` y `fprintf`) mediante un diseño modular basado en funciones.

Se sigue la metodología de programación estructurada en C, donde cada punto del taller se resuelve mediante funciones independientes que reciben parámetros validados y retornan resultados, manteniendo el `main` exclusivamente para la interacción con el usuario y la invocación de dichas funciones.

2. Punto 1: Aproximación de π con la serie de Leibniz

2.1. Descripción del problema

Aproximar el valor de π mediante la serie de Leibniz:

$$\pi \approx 4 \sum_{k=0}^{N} \frac{(-1)^k}{2k+1} = 4\left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots\right)$$

El usuario ingresa el número de términos N y el programa calcula la aproximación correspondiente.

2.2. Funciones implementadas

Función	**Entrada**	**Retorno**	**Descripción**
`calcular_pi`	`int N`	`double`	Calcula la aproximación de π sumando $N+1$ términos de la serie de Leibniz. Alterna suma y resta según la paridad de k.

Cuadro 1: Funciones del Punto 1

2.3. Validación de datos

- Se verifica que $N \geq 0$. Si el usuario ingresa un valor negativo, se imprime un mensaje de error y el programa termina de forma controlada.

2.4. Código fuente

```c
#include <stdio.h>

double calcular_pi(int N){
```

```c
    // Definir el acumulador
    double acumulador = 0.0;

    // Definir el número de iteraciones
    int k;

    // Para cada valor de K desde 0 hasta N
    for(k=0; k<= N; k++){
        // Denominador de la serie de Leibniz
        double termino = 1.0 / (2 * k + 1);

        // Revisar si es par para sumar
        if(k%2 == 0){
            acumulador = acumulador + termino;
        }else // Si es impar se resta
        acumulador = acumulador - termino;

    }
    // Se múltiplica por 4 porque es lo que está antes de la sumatoria
    return 4.0 * acumulador;
};

int main(void){

    int N;

    double resultado;

    // Solicitar entrada al usario
    printf("Ingrese el número de términos N: ");
    scanf("%d", &N);

    // Validar la entrada del usuario
    if(N < 0){
        printf("\nERROR: N debe ser mayor o igual a 0");
        return 0;
    }

    // Llamar la función
    resultado = calcular_pi(N);

    // Imprimir resultado con los 8 decimales
    printf("Aproximación de π: %.8f\n", resultado);

    return 0;
}
```

Listing 1: punto1.c – Aproximación de π

2.5. Casos de prueba

Entrada (N)	**Salida (Aproximación de π)**	**Observación**
0	4.00000000	Solo el primer término
1	2.66666667	Dos términos
10	3.23231580	Convergencia lenta
100	3.15149340	Mejor aproximación
1000	3.14259165	Cercano a π
-5	ERROR: N debe ser ≥ 0	Validación de entrada

Cuadro 2: Casos de prueba del Punto 1

3. Punto 2: Cifrado César

3.1. Descripción del problema

Implementar el cifrado César sobre una cadena de caracteres. El usuario ingresa una frase y un valor de desplazamiento d, y el programa sustituye cada letra por la que se encuentra d posiciones más adelante en el alfabeto (con envolvimiento circular). Los caracteres que no son letras (espacios, números, signos) se mantienen sin cambio.

$$c' = \big((c - \text{base}) + d\big) \text{ mód } 26 + \text{base}$$

donde `base` es `'A'` o `'a'` según el caso.

3.2. Funciones implementadas

Función	**Entrada**	**Retorno**	**Descripción**
`cifrar_cadena`	`char texto[], int desplazamiento`	`void`	Recorre la cadena carácter a carácter y aplica el desplazamiento César solo a letras mayúsculas y minúsculas. Modifica el arreglo in-place.

Cuadro 3: Funciones del Punto 2

3.3. Validación de datos

- Se soportan desplazamientos negativos: si el resultado del módulo es negativo, se suma 26 para mantener el rango válido.
- Los caracteres no alfabéticos (espacios, números, signos) no se modifican.
- Se usa `fgets` para leer la frase (evita desbordamiento) y se elimina el salto de línea residual.

3.4. Código fuente

```c
#include <stdio.h>

void cifrar_cadena(char texto[], int desplazamiento){

    int i = 0;

    // Mientras sigan habiendo caracteres en la cadena de texto...
    while(texto[i] != '\0'){
        // Revisa que este en el rango de las mayúsculas de la A a la Z
        if(texto[i] >= 'A' && texto[i] <= 'Z')
        {
            // Encontrar la posición con respecto a la A (como pivote)
            int pos = texto[i] - 'A';
            // Toma la letra y le suma el desplazamiento
            pos = (pos + desplazamiento) % 26; // El %26 es para que siempre quede la posición entre 0 y 25
            if(pos<0){
                pos = pos + 26;
            }
            texto[i] = 'A' + pos;
        }else if(texto[i] >= 'a' && texto[i] <= 'z'){
            // Lo mismo pero con las minúsculas
            int pos = texto[i] - 'a';
            pos = (pos + desplazamiento) % 26;
            if (pos < 0)
                pos = pos + 26;
            texto[i] = 'a' + pos;

        }
        // Siguiente letra
        i++;

    }


};

int main(void)
{
    // Almacenar la frase
    char texto[256];

    // Cantidad de desplazamientos
    int desplazamiento;

    // Pedirle al usuario la frase
    printf("Ingrese la frase: ");
    fgets(texto, 256, stdin);

    // Quitarle el '\n' al fgets */
    int j = 0;
    while (texto[j] != '\0')
```

```c
    {
        if (texto[j] == '\n')
        {
            texto[j] = '\0';
            break;
        }
        j++;
    }

    // Pedirle el desplazamiento
    printf("Ingrese el desplazamiento: ");
    scanf("%d", &desplazamiento);

    // Llamar la función e imprimir el texto
    cifrar_cadena(texto, desplazamiento);
    printf("Texto cifrado: %s\n", texto);

    return 0;
}
```

Listing 2: punto2.c – Cifrado César

3.5. Casos de prueba

Frase	**Despl.**	**Salida**	**Observación**
Hola Mundo	3	Krod Pxqgr	Desplazamiento positivo
Krod Pxqgr	−3	Hola Mundo	Descifrado inverso
ABC xyz	1	BCD yza	Envolvimiento z→a
Hola 123!	5	Mtqf 123!	No altera no-letras

Cuadro 4: Casos de prueba del Punto 2

4. Punto 3: Filtro de promedio móvil (suavizado de señal)

4.1. Descripción del problema

Implementar un filtro de promedio móvil que suavice un arreglo de datos simulando una señal con ruido. Dado un arreglo de entrada y un tamaño de ventana W, cada elemento de salida se calcula como el promedio de los últimos W elementos disponibles (o menos, si no hay suficientes hacia la izquierda).

$$\text{salida}[i] = \frac{1}{|\text{ventana}|} \sum_{j=\text{inicio}}^{i} \text{entrada}[j]$$

donde $\text{inicio} = \text{máx}(0,\ i - W + 1)$.

4.2. Funciones implementadas

Función	Entrada	Retorno	Descripción
`filtro_promedio`	`float entrada[]`, `float salida[]`, `int tamano`, `int ventana`	`void`	Aplica un filtro de promedio móvil sobre el arreglo `entrada` y escribe el resultado suavizado en `salida`.

Cuadro 5: Funciones del Punto 3

4.3. Validación de datos

- Se verifica que `ventana` y `tamano` sean mayores que 0. Si alguno es inválido, se imprime un mensaje de error y la función retorna sin modificar el arreglo de salida.

4.4. Código fuente

```c
#include <stdio.h>


void filtro_promedio(float entrada[], float salida[], int tamano, int
    ventana)
{
    // Revisar parametros inválidos
    if (ventana <= 0 || tamano <= 0)
    {
        printf("Error: ventana y tamano deben ser mayores que 0.\n");
        return;
    }

    // Para cada posición i del arreglo
    for (int i = 0; i < tamano; i++)
    {
        // Índice donde empieza la ventana
        int inicio;
        // Al inicio no hay suficientes elementos a la izquierda
        if (i - ventana + 1 < 0)
            inicio = 0;
        else
            inicio = i - ventana + 1;

        // Cuántos elementos se promedian (desde inicio hasta i)
        int cantidad = i - inicio + 1;
        // Acumulador de la suma
        float suma = 0.0f;

        // Sumar todos los valores de la ventana
        for (int j = inicio; j <= i; j++)
            suma += entrada[j];

```

```c
        // Promedio = suma / cantidad; se guarda en salida[i]
        salida[i] = suma / (float)cantidad;
    }
}

int main(void)
{
    // Arreglo de entrada con ruido (25.0 es el pico)
    float entrada[] = {10.2f, 10.5f, 25.0f, 10.3f, 10.1f};
    // Arreglo donde se escribe el resultado
    float salida[5];
    // Cantidad de elementos
    int tamano = 5;
    // Tamaño de la ventana del promedio
    int ventana = 3;

    // Aplicar la función para el filtro
    filtro_promedio(entrada, salida, tamano, ventana);

    // Imprimir arreglo original
    printf("Arreglo original: ");
    for (int i = 0; i < tamano; i++)
        printf("%.1f ", entrada[i]);
    printf("\n");

    // Imprimir arreglo suavizado
    printf("Arreglo suavizado: ");
    for (int i = 0; i < tamano; i++)
        printf("%.1f ", salida[i]);
    printf("\n");

    return 0;
}
```

Listing 3: punto3.c – Filtro de promedio móvil

4.5. Casos de prueba

Entrada	**W**	**Salida (suavizada)**	**Observación**
{10.2, 10.5, 25.0, 10.3, 10.1}	3	{10.2, 10.4, 15.2, 15.3, 15.1}	El pico 25.0 se atenúa
{10.2, 10.5, 25.0, 10.3, 10.1}	1	{10.2, 10.5, 25.0, 10.3, 10.1}	Sin suavizado
{10.2, 10.5, 25.0, 10.3, 10.1}	5	{10.2, 10.4, 15.2, 14.0, 13.2}	Máximo suavizado
Cualquiera	0	Error	Validación de ventana

Cuadro 6: Casos de prueba del Punto 3

5. Punto 4: Detección de picos en una matriz 2D

5.1. Descripción del problema

Dada una matriz de enteros de 4×4 que representa un mapa de valores (por ejemplo, intensidades de señal), detectar todos los *picos*: celdas cuyo valor es estrictamente mayor que el de sus cuatro vecinos ortogonales (arriba, abajo, izquierda, derecha). Las celdas en los bordes tienen menos vecinos, lo cual se maneja verificando los límites antes de comparar.

5.2. Funciones implementadas

Función	Entrada	Retorno	Descripción
`detectar_picos`	`int filas, int cols, int mapa[4][4]`	`void`	Recorre la matriz comparando cada celda con sus vecinos. Almacena las coordenadas de los picos encontrados y los imprime.

Cuadro 7: Funciones del Punto 4

5.3. Validación de datos

- Antes de acceder a cada vecino se verifica que el índice esté dentro de los límites de la matriz (`i-1 >= 0`, `j+1 <cols`, etc.).
- Si no se encuentran picos, se imprime un mensaje informativo.

5.4. Código fuente

```c
#include <stdio.h>

#define MAX_CELDAS 16

void detectar_picos(int filas, int cols, int mapa[4][4]);

int main(void)
{
    // Armamos el mapa 4x4; el 9 en el medio es el pico
    int mapa[4][4] = {
        {1, 2, 1, 0},
        {2, 9, 2, 0},
        {1, 2, 1, 0},
        {0, 0, 0, 0}
    };

    //Llamar la funcipon
    detectar_picos(4, 4, mapa);

    return 0;
```

```c
}

void detectar_picos(int filas, int cols, int mapa[4][4])
{
    // Para guardar las coordenadas de cada pico que encontremos
    int picos_i[MAX_CELDAS];
    int picos_j[MAX_CELDAS];
    int num_picos = 0;

    // Recorremos toda la matriz, celda por celda
    for (int i = 0; i < filas; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            int es_pico = 1;  // De entrada pues se asume, y si algún vecino lo supera lo bajamos

            // arriba: solo miro si existe esa fila
            if (i - 1 >= 0 && mapa[i][j] <= mapa[i - 1][j])
                es_pico = 0;
            // abajo
            if (i + 1 < filas && mapa[i][j] <= mapa[i + 1][j])
                es_pico = 0;
            // izquierda
            if (j - 1 >= 0 && mapa[i][j] <= mapa[i][j - 1])
                es_pico = 0;
            // derecha
            if (j + 1 < cols && mapa[i][j] <= mapa[i][j + 1])
                es_pico = 0;

            // si pasó todas las pruebas, anotamos el par (i, j)
            if (es_pico)
            {
                picos_i[num_picos] = i;
                picos_j[num_picos] = j;
                num_picos++;
            }
        }
    }

    // si no encontramos nada, avisamos y seguimos
    if (num_picos == 0)
    {
        printf("No se encontraron picos.\n");
        return;
    }

    // listamos todos los picos que fuimos guardando
    for (int k = 0; k < num_picos; k++)
        printf("Pico en (%d, %d) con valor %d\n", picos_i[k], picos_j[k], mapa[picos_i[k]][picos_j[k]]);
}
```

Listing 4: punto4.c – Detección de picos en matriz 2D

5.5. Casos de prueba

Matriz de entrada	Salida	Observación
{1,2,1,0}, {2,9,2,0}, {1,2,1,0}, {0,0,0,0}	Pico en (1,1) valor 9	Mayor que sus 4 vecinos
Matriz con todos los valores iguales	No se encontraron picos	Ningún valor es estrictamente mayor

Cuadro 8: Casos de prueba del Punto 4

6. Punto 5: Lectura de archivo con estructuras

6.1. Descripción del problema

Leer un archivo de texto (`sensores.txt`) que contiene mediciones de sensores en formato CSV (`ID,temperatura,humedad`) y almacenar cada registro en un arreglo de estructuras `Medicion`. Luego mostrar los datos cargados por pantalla.

Formato del archivo:

```
SENS_01,24.5,60
SENS_02,22.3,55
SENS_03,25.0,62
```

6.2. Funciones implementadas

Función	Entrada	Retorno	Descripción
`cargar_datos`	`char nombre_archivo[]`, `Medicion datos[]`, `int max_datos`	`int`	Abre el archivo, lee línea a línea con `fgets` y extrae los campos con `sscanf`. Retorna la cantidad de registros leídos, o -1 si el archivo no se pudo abrir.

Cuadro 9: Funciones del Punto 5

6.3. Estructura de datos

```
typedef struct {
    char id[32];
    float temperatura;
    int humedad;
} Medicion;
```

6.4. Validación de datos

- Si `fopen` retorna `NULL`, se imprime un mensaje de error y se retorna -1 sin colapsar.
- No se leen más de `max_datos` registros para evitar desbordamiento del arreglo.
- Solo se guardan líneas donde `sscanf` parsea exactamente 3 campos.

6.5. Código fuente

```c
#include <stdio.h>

// Estructura para guardar una medición (id, temperatura, humedad)
typedef struct {
    // ID del sensor, ej. SENS_01
    char id[32];
    // Temperatura en grados
    float temperatura;
    // Humedad en porcentaje
    int humedad;
} Medicion;

// Carga las mediciones desde el archivo y devuelve cuántas se leyeron
int cargar_datos(char nombre_archivo[], Medicion datos[], int max_datos);

int main(void)
{
    // Arreglo donde se guardan las mediciones leídas
    Medicion datos[64];
    // Tamaño máximo del arreglo para no pasarse
    int max_datos = 64;
    // Cantidad de registros que devolvió cargar_datos
    int n;

    n = cargar_datos("sensores.txt", datos, max_datos);

    // Si retornó -1, hubo error al abrir el archivo
    if (n < 0)
        return 1;

    // Mostrar cuántas se cargaron y listar cada una
    printf("Se cargaron %d mediciones:\n", n);
    for (int i = 0; i < n; i++)
        printf("  %s - Temperatura: %.1f, Humedad: %d\n", datos[i].id,
    datos[i].temperatura, datos[i].humedad);

    return 0;
}

int cargar_datos(char nombre_archivo[], Medicion datos[], int max_datos)
{
    FILE *f;
    // Buffer para leer cada línea del archivo
    char linea[128];
```

```c
    // Contador de mediciones guardadas
    int num = 0;

    f = fopen(nombre_archivo, "r");
    if (f == NULL)
    {
        printf("Error: no se pudo abrir el archivo '%s'.\n",
    nombre_archivo);
        return -1;
    }

    // Leer línea a línea hasta que no haya más
    while (fgets(linea, sizeof(linea), f) != NULL)
    {
        // No guardar más de max_datos
        if (num >= max_datos)
            break;

        // Extraer id,temp,hum con sscanf; formato: ID,float,int
        // Si sscanf lee los 3 campos retorna 3 y guardamos en datos[num]
        if (sscanf(linea, "%[^,],%f,%d", datos[num].id, &datos[num].
    temperatura, &datos[num].humedad) == 3)
            num++;
    }

    fclose(f);
    return num;
}
```

Listing 5: punto5.c – Lectura de archivo con estructuras

6.6. Casos de prueba

Archivo	Salida	Observación
`sensores.txt` (3 líneas válidas)	Se cargaron 3 mediciones: SENS_01 24.5 60, SENS_02 22.3 55, SENS_03 25.0 62	Lectura correcta
Archivo inexistente	Error: no se pudo abrir el archivo	Manejo de `fopen` NULL

Cuadro 10: Casos de prueba del Punto 5

7. Punto 6: Análisis de temperaturas

7.1. Descripción del problema

A partir de los datos cargados desde `sensores.txt` (reutilizando `cargar_datos` del Punto 5), determinar la temperatura máxima, la mínima y el promedio general, indicando el ID del sensor correspondiente para los extremos.

7.2. Funciones implementadas

Función	Entrada	Retorno	Descripción
analizar_temperaturas	Medicion datos[], int cantidad	void	Recorre el arreglo de mediciones para encontrar la temperatura máxima, mínima y el promedio. Imprime los resultados con el ID del sensor.
cargar_datos	(igual que Punto 5)	int	Reutilizada del Punto 5.

Cuadro 11: Funciones del Punto 6

7.3. Validación de datos

- Si cantidad es ≤ 0, se imprime "No hay datos para analizar" y la función retorna sin operar (evita división por cero en el promedio).

7.4. Código fuente

```c
#include <stdio.h>

// Estructura para una medición (igual que punto 5)
typedef struct {
    char id[32];
    float temperatura;
    int humedad;
} Medicion;

// Firmes con la patria jajaja (firma de las funciones)
int cargar_datos(char nombre_archivo[], Medicion datos[], int max_datos);
void analizar_temperaturas(Medicion datos[], int cantidad);

int main(void)
{
    Medicion datos[64];
    int max_datos = 64;
    int n;

    n = cargar_datos("sensores.txt", datos, max_datos);
    if (n < 0)
        return 1;

    // Analizar el arreglo llenado y mostrar máx, mín y promedio
    analizar_temperaturas(datos, n);

    return 0;
}

void analizar_temperaturas(Medicion datos[], int cantidad)
{
```

```c
    // Evitar división por cero y arreglo vacío
    if (cantidad <= 0)
    {
        printf("No hay datos para analizar.\n");
        return;
    }

    // Inicializar con el primer elemento
    float temp_max = datos[0].temperatura;
    float temp_min = datos[0].temperatura;
    float suma = 0.0f;
    int indice_max = 0;
    int indice_min = 0;

    // Recorrer el arreglo para obtener máx, mín y suma
    for (int i = 0; i < cantidad; i++)
    {
        suma += datos[i].temperatura;
        if (datos[i].temperatura > temp_max)
        {
            temp_max = datos[i].temperatura;
            indice_max = i;
        }
        if (datos[i].temperatura < temp_min)
        {
            temp_min = datos[i].temperatura;
            indice_min = i;
        }
    }

    // Promedio = suma / cantidad
    float promedio = suma / (float)cantidad;

    // Imprimir resultados con el ID del sensor para máx y mín
    printf("Temperatura maxima: %.1f (sensor %s)\n", temp_max, datos[indice_max].id);
    printf("Temperatura minima: %.1f (sensor %s)\n", temp_min, datos[indice_min].id);
    printf("Promedio general: %.1f\n", promedio);
}

int cargar_datos(char nombre_archivo[], Medicion datos[], int max_datos)
{
    FILE *f;
    char linea[128];
    int num = 0;

    f = fopen(nombre_archivo, "r");
    if (f == NULL)
    {
        printf("Error: no se pudo abrir el archivo '%s'.\n", nombre_archivo);
        return -1;
    }
```

```c

    while (fgets(linea, sizeof(linea), f) != NULL)
    {
        if (num >= max_datos)
            break;
        if (sscanf(linea, "%[^,],%f,%d", datos[num].id, &datos[num].temperatura, &datos[num].humedad) == 3)
            num++;
    }

    fclose(f);
    return num;
}
```

Listing 6: punto6.c – Análisis de temperaturas

7.5. Casos de prueba

Archivo	Salida	Observación
`sensores.txt` (3 sensores)	Máxima: 25.0 (SENS_03), Mínima: 22.3 (SENS_02), Promedio: 23.9	Cálculos correctos
Archivo vacío o inexistente	Error o "No hay datos"	Validación de entrada

Cuadro 12: Casos de prueba del Punto 6

8. Punto 7: Exportar datos corregidos a archivo

8.1. Descripción del problema

Cargar las mediciones desde `sensores.txt`, aplicar un *offset* de calibración a todas las temperaturas y escribir los datos corregidos en un nuevo archivo (`sensores_calibrados.txt`) con el mismo formato CSV.

8.2. Funciones implementadas

Función	Entrada	Retorno	Descripción
`exportar_corregido`	`char nom_arch[]`, `Medicion datos[]`, `int cant`, `float offset`	`void`	Crea un archivo de salida y escribe cada medición con la temperatura corregida (`temp + offset`) usando `fprintf`.
`cargar_datos`	(igual que Punto 5)	`int`	Reutilizada del Punto 5.

Cuadro 13: Funciones del Punto 7

8.3. Validación de datos

- Si `fopen` para escritura retorna `NULL`, se imprime un mensaje de error y la función retorna sin colapsar.
- Se verifica el retorno de `cargar_datos` antes de proceder.

8.4. Código fuente

```c
#include <stdio.h>

// Estructura para una medición (igual que punto 5)
typedef struct {
    char id[32];
    float temperatura;
    int humedad;
} Medicion;

int cargar_datos(char nombre_archivo[], Medicion datos[], int max_datos);
void exportar_corregido(char nom_arch[], Medicion datos[], int cant, float offset);

int main(void)
{
    Medicion datos[64];
    int max_datos = 64;
    int n;

    n = cargar_datos("sensores.txt", datos, max_datos);
    if (n < 0)
        return 1;

    // Corregir temperaturas con offset y guardar en archivo
    exportar_corregido("sensores_calibrados.txt", datos, n, -1.5f);

    return 0;
}

void exportar_corregido(char nom_arch[], Medicion datos[], int cant, float offset)
{
    FILE *f;

    f = fopen(nom_arch, "w");
    if (f == NULL)
    {
        printf("Error: no se pudo crear el archivo '%s'.\n", nom_arch);
        return;
    }

    // Recorrer el arreglo y escribir cada línea con la temperatura corregida
    for (int i = 0; i < cant; i++)
    {
```

```c
        float nueva_temp = datos[i].temperatura + offset;
        fprintf(f, "%s,%.1f,%d\n", datos[i].id, nueva_temp, datos[i].humedad);
    }

    fclose(f);
}

int cargar_datos(char nombre_archivo[], Medicion datos[], int max_datos)
{
    FILE *f;
    char linea[128];
    int num = 0;

    f = fopen(nombre_archivo, "r");
    if (f == NULL)
    {
        printf("Error: no se pudo abrir el archivo '%s'.\n", nombre_archivo);
        return -1;
    }

    while (fgets(linea, sizeof(linea), f) != NULL)
    {
        if (num >= max_datos)
            break;
        if (sscanf(linea, "%[^,],%f,%d", datos[num].id, &datos[num].temperatura, &datos[num].humedad) == 3)
            num++;
    }

    fclose(f);
    return num;
}
```

Listing 7: punto7.c – Exportar datos corregidos

8.5. Casos de prueba

Offset	Temp. original	Temp. corregida	Observación
−1,5	24.5, 22.3, 25.0	23.0, 20.8, 23.5	Genera archivo calibrado
+2,0	24.5, 22.3, 25.0	26.5, 24.3, 27.0	Offset positivo
0,0	24.5, 22.3, 25.0	24.5, 22.3, 25.0	Sin cambio

Cuadro 14: Casos de prueba del Punto 7

9. Conclusiones

- Se aplicó el diseño modular separando la lógica en funciones independientes, manteniendo el `main` limpio y enfocado en la interacción con el usuario.
- Se implementó validación de datos en todas las entradas del usuario y en los parámetros de las funciones, garantizando que el programa no colapse ante valores inválidos.
- Se manipularon cadenas de caracteres mediante aritmética de posiciones ASCII para implementar el cifrado César, con soporte para desplazamientos negativos.
- Se trabajó con arreglos 1D para el filtro de promedio móvil y con arreglos 2D para la detección de picos, aplicando verificación de límites en los accesos a vecinos.
- Se utilizaron estructuras (`struct`) para organizar datos de sensores de forma coherente, agrupando ID, temperatura y humedad en un solo tipo de dato.
- Se practicó la persistencia de datos mediante lectura y escritura de archivos con `fopen`, `fgets`, `sscanf` y `fprintf`, verificando siempre el retorno de `fopen` para evitar errores fatales.
- Se demostró la reutilización de funciones al emplear `cargar_datos` en los puntos 5, 6 y 7, evidenciando las ventajas del diseño modular.